

2 Practice and Comprehensive Questions Solutions

```
import numpy as np
import numpy.random as random
import numpy.linalg as LA

# Display up to 3 decimal places
%precision 3

'%.3f'
```

<Practice Question 2-1>

(1) Create two 1-dimensional `np.ndarray` arrays with your favorite values as elements, and use the `type()` function to confirm that they are of type `np.ndarray`.

```
# (1)
array1 = np.array([1, 2, 3, 4, 5])
array2 = np.array([6, 7, 8, 9, 10])
```

(2) Perform arithmetic operations on the two 1-dimensional arrays created in (1).

```
type(array1)
```

```
numpy.ndarray
```

```
type(array2)
```

```
numpy.ndarray
```

```
# (2)
print('Addition', array1 + array2)
print('Substraction', array1 - array2)
print('Multiplication', array1 * array2)
print('Division', array1 / array2)
```

```
Addition [ 7  9 11 13 15]
Substraction [-5 -5 -5 -5 -5]
Multiplication [ 6 14 24 36 50]
Division [0.167 0.286 0.375 0.444 0.5 ]
```

(3) Create a function that takes a 1-dimensional array as input and outputs a 1-dimensional array where the input is normalized.

Here, normalization of a vector (using the L2 norm) means scaling the original vector so that the square root of the sum of the squares of all its elements equals 1.

In this task, please implement the function using only **Numpy's universal operations and functions**.

```
def normalize(a):
    return a / np.sqrt(np.sum(a**2))
```

```
array1
```

```
array([1, 2, 3, 4, 5])
```

```
print(normalize(array1))
print(np.sqrt(np.sum(normalize(array1)**2)))
```

```
[0.135 0.27  0.405 0.539 0.674]
1.0
```

<Practice Question 2-2>

(1) For any array you create, retrieve and display the first value from the front and the first value from the back.

```
a = np.array([1, 2, 3, 4, 5])
print("The first value from the front:", a[0])
print("The second value from the back:", a[-1])
```

```
The first value from the front: 1
The second value from the back: 5
```

(2) Implement a function that takes a one-dimensional Numpy array with integer values as an argument and outputs an array containing elements that are either odd (leave a remainder of 1 when divided by 2) or leave a remainder of 2 when divided by 4.

```
def make_bool_ids(a):
    idx1 = (a % 2 != 0)
    idx2 = (a % 4 == 2)
    idx = idx1 | idx2
    return a[idx]
```

```
# Input Example
a = np.array([1, 1, 2, 2, 3, 3, 4, 4, 5, 5])
print(make_bool_ids(a))
# Output Example
# np.array([1, 1, 2, 2, 3, 3, 5, 5])
```

```
[1 1 2 2 3 3 5 5]
```

(3) Implement a function that takes a positive integer as an argument and counts the number of perfect squares less than the given number using boolean indexing. (Hint: List enough perfect squares in ascending order and count those smaller than the argument.)

```
def count_square(n):
    array = np.arange(1, n) ** 2
    answer = np.sum((array < n))
    return answer
```

```
# Alternative Solution
def count_square(n):
    array = np.arange(1, n)
    answer = np.sum((array < np.sqrt(n)))
    return answer
```

```
# Input Example
n = 20
answer = count_square(n)
print(answer)
# Output Example
# The positive perfect squares less than 20 are 1, 4, 9, and 16.
# 4
```

```
4
```

▼ <Practice Question 2-3>

(1) Store the test results for 5 students over 4 test sessions in a 2-dimensional array, with 100 points as the maximum score. Here, axis 0 corresponds to the students, and axis 1 corresponds to the test sessions. The values can be arbitrary.

```
array = np.array([
    [90, 100, 80, 56],
    [67, 94, 29, 90],
    [94, 56, 92, 74],
    [39, 46, 62, 74],
    [23, 48, 81, 83]
])
```

(2) For each student, calculate the highest score, lowest score, average, and variance across the 4 test sessions. Store the results in a 2-dimensional array where axis 0 corresponds to the students and axis 1 corresponds to the aggregation results (highest score, lowest score, average, variance).

```
print('student1',
      f'Highest Score: {np.max(array, axis=1)[0]}',
      f'Lowest Score: {np.min(array, axis=1)[0]}',
      f'Average Score: {np.mean(array, axis=1)[0]}',
      f'Variance: {np.var(array, axis=1)[0]}')
)

print('student2',
      f'Highest Score: {np.max(array, axis=1)[1]}',
      f'Lowest Score: {np.min(array, axis=1)[1]}',
      f'Average Score: {np.mean(array, axis=1)[1]}',
      f'Variance: {np.var(array, axis=1)[1]}')
)

print('student3',
      f'Highest Score: {np.max(array, axis=1)[2]}',
      f'Lowest Score: {np.min(array, axis=1)[2]}',
      f'Average Score: {np.mean(array, axis=1)[2]}',
      f'Variance: {np.var(array, axis=1)[2]}')
)

print('student4',
      f'Highest Score: {np.max(array, axis=1)[3]}',
      f'Lowest Score: {np.min(array, axis=1)[3]}',
      f'Average Score: {np.mean(array, axis=1)[3]}',
      f'Variance: {np.var(array, axis=1)[3]}')
)

print('student5',
      f'Highest Score: {np.max(array, axis=1)[4]}',
      f'Lowest Score: {np.min(array, axis=1)[4]}',
      f'Average Score: {np.mean(array, axis=1)[4]}',
      f'Variance: {np.var(array, axis=1)[4]}')
)

array2 = np.array([
    [np.max(array, axis=1)[0], np.min(array, axis=1)[0], np.mean(array, axis=1)[0], np.var(array, axis=1)[0]],
    [np.max(array, axis=1)[1], np.min(array, axis=1)[1], np.mean(array, axis=1)[1], np.var(array, axis=1)[1]],
    [np.max(array, axis=1)[2], np.min(array, axis=1)[2], np.mean(array, axis=1)[2], np.var(array, axis=1)[2]],
    [np.max(array, axis=1)[3], np.min(array, axis=1)[3], np.mean(array, axis=1)[3], np.var(array, axis=1)[3]],
    [np.max(array, axis=1)[4], np.min(array, axis=1)[4], np.mean(array, axis=1)[4], np.var(array, axis=1)[4]],
])
```

```
student1 Highest Score: 100 Lowest Score: 56 Average Score: 81.5 Variance: 266.75
student2 Highest Score: 94 Lowest Score: 29 Average Score: 70.0 Variance: 666.5
student3 Highest Score: 94 Lowest Score: 56 Average Score: 79.0 Variance: 237.0
student4 Highest Score: 74 Lowest Score: 39 Average Score: 55.25 Variance: 186.6875
student5 Highest Score: 83 Lowest Score: 23 Average Score: 58.75 Variance: 619.1875
```

array2

```
array([[100. , 56. , 81.5 , 266.75 ],
       [ 94. , 29. , 70. , 666.5 ],
       [ 94. , 56. , 79. , 237. ],
       [ 74. , 39. , 55.25 , 186.688 ],
       [ 83. , 23. , 58.75 , 619.188 ]])
```

array2[:,2].reshape(5,1)

```
array([[81.5 ],
       [70. ],
       [79. ],
       [55.25],
       [58.75]])
```

(3) For each test score, calculate the absolute difference between the score and the student's average score, resulting in a 2-dimensional array with shape (5, 4).

```
array3 = np.abs(array - array2[:,2].reshape(5,1))
```

```
print(array3)
print(array3.shape)

[[ 8.5  18.5   1.5  25.5 ]
 [ 3.   24.   41.   20. ]
 [15.   23.   13.    5. ]
 [16.25  9.25  6.75 18.75]
 [35.75 10.75 22.25 24.25]]
(5, 4)
```

▼ <Practice Question 2-4>

(1) Create a 1-dimensional np.ndarray with 100 consecutive integers from 0 to 99, and reshape it into a 2-dimensional array with shape (10, 10).

```
array1 = np.arange(0,100).reshape(10,10)
array1

array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
       [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
       [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
       [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
       [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
       [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
       [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
       [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
```

(2) From the array created in (1), extract the even rows (using 0-based indexing), resulting in a 2-dimensional array with shape (5, 10).

```
idx1 = np.arange(0, array1.shape[0], 2)
array1[idx1, :]

array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
       [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
       [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
       [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
       [80, 81, 82, 83, 84, 85, 86, 87, 88, 89]])
```

(3) From the array created in (1), extract the odd columns (using 0-based indexing), resulting in a 2-dimensional array with shape (10, 5).

```
idx2 = np.arange(1, array1.shape[1], 2)
array1[:,idx2]

array([[ 1,  3,  5,  7,  9],
       [11, 13, 15, 17, 19],
       [21, 23, 25, 27, 29],
       [31, 33, 35, 37, 39],
       [41, 43, 45, 47, 49],
       [51, 53, 55, 57, 59],
       [61, 63, 65, 67, 69],
       [71, 73, 75, 77, 79],
       [81, 83, 85, 87, 89],
       [91, 93, 95, 97, 99]])
```

(4) From the array created in (1), extract the values in even rows and odd columns (using 0-based indexing), resulting in a 2-dimensional array with shape (5, 5).

```
array1[np.ix_(idx1, idx2)]

array([[ 1,  3,  5,  7,  9],
       [21, 23, 25, 27, 29],
       [41, 43, 45, 47, 49],
       [61, 63, 65, 67, 69],
       [81, 83, 85, 87, 89]])
```

2.7 Comprehensive Question

Question 1

Recently, in natural language processing using deep learning, models based on Google's Transformer have gained attention due to their human-level performance on several NLP tasks.

In January 2020, Google introduced [Reformer](#), which improves the time and space complexity of the Transformer model. In this task, we will implement a part of the Reformer paper, which uses [cross-polytope LSH](#) to improve the time complexity of the Transformer, with guided instructions.

Consider the following question:

Given n vectors, we want to assign the same integer value to "similar" vectors and different integer values to "dissimilar" vectors. In this context, "similar" vectors are those whose angle is close to 0.

The challenges of this problem are as follows:

- How small should the angle be for vectors to be considered "similar"?
- Even if vectors a and b , and b and c are "similar", a and c may not necessarily be considered "similar."
- To compute the angle θ , we can use their inner product and norms to calculate $\cos \theta$. We could then sort this value and assign integers accordingly, but sorting can become slow as n increases (since the number of combinations of vectors is ${}_nC_2 = O(n^2)$).

We consider the following probabilistic approach:

1. Rotate all vectors by the same random angle.
2. Vectors that are "similar" will, with **high probability**, remain in the same quadrant after rotation, while vectors that are "dissimilar" will, with **high probability**, end up in different quadrants.
3. Therefore, by assigning each vector an integer value corresponding to the quadrant it belongs to after the random rotation, the problem can be solved with **high probability**.

Let's implement this approach.

This time, we consider 100 3D row vectors.

```
A = random.randn(100, 3) # A matrix stacked with 100 3D row vectors
print(A)
```

```
[[-2.328e-01  1.651e-01  5.701e-01]
 [ 8.563e-01 -1.035e+00 -1.340e+00]
 [-2.893e-01 -2.034e+00 -9.624e-01]
 [ 1.346e+00  1.008e+00 -1.355e+00]
 [-1.437e+00 -1.148e+00 -2.892e-02]
 [ 2.983e-01  4.697e-01  1.666e+00]
 [ 1.737e+00 -7.840e-03  5.393e-01]
 [-3.187e-01 -1.760e-01 -1.086e+00]
 [ 8.293e-01 -7.569e-01  8.159e-01]
 [ 2.735e-01 -2.805e-01  7.136e-01]
 [-5.717e-01 -1.933e-01  2.895e-01]
 [-4.230e-01  1.856e+00 -8.716e-01]
 [ 5.592e-01 -1.056e+00 -7.764e-01]
 [-3.821e-01  6.392e-01  1.886e+00]
 [ 2.793e-01 -8.803e-01 -1.608e-01]
 [ 3.298e-02 -1.168e-01  3.242e-01]
 [ 5.433e-01 -1.916e-01 -3.634e-01]
 [ 1.820e-01  1.863e-01  1.528e+00]
 [-4.593e-02  1.826e+00  2.893e-01]
 [-8.259e-02 -2.848e+00 -5.059e-01]
 [ 2.441e-01 -5.875e-01 -4.847e-01]
 [ 2.160e-01 -9.862e-01 -6.170e-01]
 [ 2.461e+00  1.250e+00  8.463e-01]
 [-1.126e+00 -8.301e-01 -1.385e+00]
 [-9.009e-01 -3.469e-01  4.102e-01]
 [-1.924e+00 -3.529e-01 -1.730e-01]
 [ 8.731e-01  7.948e-01  1.128e+00]
 [ 1.784e-01 -1.202e+00 -1.208e+00]
 [ 2.083e+00 -1.818e+00 -3.063e-02]
 [ 4.658e-01  1.516e-01  1.406e+00]
 [ 1.233e+00 -2.450e-01  1.736e-01]
 [-1.084e+00 -2.938e-01 -5.830e-01]
 [ 6.021e-01 -9.281e-01 -8.890e-01]
```

```

[-5.229e-01 -2.227e+00 1.185e+00]
[-1.715e+00 7.717e-01 -1.121e-01]
[ 1.179e+00 6.923e-01 9.569e-01]
[ 2.986e-01 -7.875e-03 -1.758e+00]
[ 1.880e+00 -8.283e-01 1.059e+00]
[ 1.030e+00 -5.521e-01 -1.228e+00]
[ 2.883e+00 1.431e+00 6.826e-02]
[-1.365e-01 -5.456e-01 -1.502e+00]
[ 1.005e+00 -8.520e-01 -1.954e-01]
[ 1.460e+00 -1.399e-01 2.622e-01]
[ 6.971e-01 2.066e-01 -9.513e-02]
[-1.770e-01 5.145e-01 1.199e+00]
[-9.955e-01 -2.130e-01 2.001e+00]
[ 6.188e-01 -2.550e-01 -8.093e-01]
[-2.255e-01 -1.163e+00 6.654e-01]
[ 1.163e-01 -1.486e+00 -5.511e-01]
[ 1.567e-01 -2.549e-01 4.362e-02]
[ 1.687e-01 3.154e-01 6.096e-01]
[ 1.078e+00 -1.407e+00 8.703e-01]
[-2.594e-01 -8.411e-01 2.769e+00]
[-1.196e+00 2.005e+00 -1.917e-01]
[ 1.128e+00 -4.778e-01 2.731e-01]
[-2.582e-01 1.162e+00 5.050e-01]
[ 1.091e+00 -5.323e-01 9.989e-01]
[ 6.021e-01 6.810e-01 2.487e+00]

```

(1) Generate a rotation matrix R that rotates 100 vectors, and compute the matrix of rotated vectors stacked vertically into a shape of (100, 3). Here, the 3D rotation matrix R for a random rotation is generated by passing a 1D array of three randomly generated angle parameters to the `gen_R_3d` function.

```

# (1)
def gen_R_3d(p):
    px, py, pz = p
    Rx = np.array([
        [1, 0, 0],
        [0, np.cos(px), np.sin(px)],
        [0, -np.sin(px), np.cos(px)]
    ])
    Ry = np.array([
        [np.cos(py), 0, -np.sin(py)],
        [0, 1, 0],
        [np.sin(py), 0, np.cos(py)]
    ])
    Rz = np.array([
        [np.cos(pz), np.sin(pz), 0],
        [-np.sin(pz), np.cos(pz), 0],
        [0, 0, 1]
    ])

    return Rz @ Ry @ Rx

p = 2 * np.pi * random.rand(3)
R = gen_R_3d(p)

B = (R @ A.T).T
print(B)

```

```

[[ 0.027  0.362 -0.524]
 [-0.731 -0.949  1.471]
 [-1.476 -1.695  0.312]
 [ 1.015  0.136  1.902]
 [-0.677 -1.263 -1.153]
 [-0.18  1.581 -0.744]
 [-0.618  1.33  1.076]
 [ 0.268 -1.044  0.387]
 [-1.162  0.74  0.171]
 [-0.554  0.559 -0.208]
 [-0.125 -0.173 -0.634]
 [ 2.08 -0.114  0.203]
 [-0.857 -0.702  0.897]
 [ 0.076  1.446 -1.42 ]
 [-0.823 -0.327  0.308]
 [-0.219  0.209 -0.168]
 [-0.196 -0.054  0.65 ]
 [-0.366  1.301 -0.759]
 [ 1.586  0.933 -0.184]
 [-2.418 -1.579  0.196]
 [-0.443 -0.466  0.476]
 [-0.757 -0.741  0.527]
 [ 0.249  2.459  1.492]

```

```

[-0.03 -1.966 -0.093]
[-0.221 -0.324 -0.973]
[ 0.219 -1.307 -1.449]
[ 0.144 1.626 0.042]
[-0.755 -1.285 0.845]
[-2.176 0.358 1.668]
[-0.43 1.35 -0.459]
[-0.59 0.692 0.885]
[ 0.192 -1.133 -0.529]
[-0.714 -0.71 1. ]
[-2.281 -0.315 -1.155]
[ 1.173 -0.692 -1.302]
[ 0.028 1.623 0.388]
[ 0.481 -1.139 1.285]
[-1.57 1.456 0.871]
[-0.371 -0.576 1.55 ]
[ 0.557 2.186 2.296]
[ 0.018 -1.404 0.776]
[-0.969 0.05 0.913]
[-0.58 0.922 1.016]
[ 0.043 0.389 0.62 ]
[ 0.13 0.999 -0.848]
[-0.585 0.853 -1.993]
[-0.13 -0.368 0.975]
[-1.218 -0.105 -0.593]
[-1.21 -0.95 0.401]
[-0.286 0.013 0.097]
[ 0.05 0.669 -0.222]
[-1.835 0.649 0.33 ]
[-1.59 1.559 -1.866]
[ 2.194 0.032 -0.821]
[-0.809 0.614 0.738]
[ 0.964 0.707 -0.492]
[-1.081 1.107 0.276]
[ 1.594 -1.929 0.931]

```

(2) To normalize each vector obtained in (1) so that the L2 norm equals 1 and then stack them vertically to form a (100, 3) matrix, you would follow these steps:

Compute the L2 norm for each vector. Normalize each vector by dividing by its L2 norm. Stack the normalized vectors without changing their order.

```

# (2)
B_norm_l2 = LA.norm(B, ord=2, axis=1, keepdims=True)
B_normalized = B / B_norm_l2
print(B_normalized)

```

```

[[ 0.042 0.568 -0.822]
[-0.386 -0.5 0.775]
[-0.65 -0.747 0.137]
[ 0.47 0.063 0.88 ]
[-0.368 -0.687 -0.627]
[-0.103 0.9 -0.424]
[-0.34 0.731 0.592]
[ 0.234 -0.912 0.338]
[-0.837 0.533 0.124]
[-0.68 0.687 -0.256]
[-0.187 -0.259 -0.948]
[ 0.994 -0.054 0.097]
[-0.601 -0.493 0.629]
[ 0.037 0.713 -0.7 ]
[-0.878 -0.349 0.329]
[-0.632 0.604 -0.485]
[-0.287 -0.079 0.955]
[-0.236 0.839 -0.49 ]
[ 0.858 0.505 -0.1 ]
[-0.835 -0.546 0.068]
[-0.553 -0.582 0.596]
[-0.64 -0.626 0.445]
[ 0.086 0.852 0.517]
[-0.015 -0.999 -0.047]
[-0.211 -0.309 -0.928]
[ 0.111 -0.666 -0.738]
[ 0.088 0.996 0.026]
[-0.441 -0.75 0.493]
[-0.787 0.129 0.603]
[-0.289 0.906 -0.308]
[-0.465 0.545 0.697]
[ 0.152 -0.896 -0.418]
[-0.503 -0.5 0.705]
[-0.885 -0.122 -0.448]
[ 0.623 -0.367 -0.691]

```

```
[ 0.017 0.972 0.233]
[ 0.27 -0.639 0.721]
[-0.679 0.63 0.377]
[-0.219 -0.34 0.915]
[ 0.173 0.679 0.713]
[ 0.011 -0.875 0.484]
[-0.727 0.037 0.685]
[-0.389 0.619 0.682]
[ 0.059 0.531 0.845]
[ 0.099 0.759 -0.644]
[-0.26 0.38 -0.888]
[-0.123 -0.35 0.929]
[-0.897 -0.077 -0.436]
[-0.761 -0.598 0.252]
[-0.947 0.042 0.319]
[ 0.071 0.947 -0.315]
[-0.93 0.329 0.167]
[-0.547 0.537 -0.642]
[ 0.937 0.014 -0.35 ]
[-0.644 0.489 0.588]
[ 0.746 0.547 -0.381]
[-0.688 0.704 0.176]
[ 0.507 -0.722 0.240]
```

(3) For the 100 vectors with a norm of 1 obtained in (2), compute a matrix of shape (100, 6) by stacking the original 100 vectors and their -1 multiplied counterparts **horizontally**, without changing the order of the vectors.

```
# (3)
C = np.concatenate([B_normalized, -B_normalized], axis=1)
print(C)
```

```
[[ 0.042 0.568 -0.822 -0.042 -0.568 0.822]
[-0.386 -0.5 0.775 0.386 0.5 -0.775]
[-0.65 -0.747 0.137 0.65 0.747 -0.137]
[ 0.47 0.063 0.88 -0.47 -0.063 -0.88 ]
[-0.368 -0.687 -0.627 0.368 0.687 0.627]
[-0.103 0.9 -0.424 0.103 -0.9 0.424]
[-0.34 0.731 0.592 0.34 -0.731 -0.592]
[ 0.234 -0.912 0.338 -0.234 0.912 -0.338]
[-0.837 0.533 0.124 0.837 -0.533 -0.124]
[-0.68 0.687 -0.256 0.68 -0.687 0.256]
[-0.187 -0.259 -0.948 0.187 0.259 0.948]
[ 0.994 -0.054 0.097 -0.994 0.054 -0.097]
[-0.601 -0.493 0.629 0.601 0.493 -0.629]
[ 0.037 0.713 -0.7 -0.037 -0.713 0.7 ]
[-0.878 -0.349 0.329 0.878 0.349 -0.329]
[-0.632 0.604 -0.485 0.632 -0.604 0.485]
[-0.287 -0.079 0.955 0.287 0.079 -0.955]
[-0.236 0.839 -0.49 0.236 -0.839 0.49 ]
[ 0.858 0.505 -0.1 -0.858 -0.505 0.1 ]
[-0.835 -0.546 0.068 0.835 0.546 -0.068]
[-0.553 -0.582 0.596 0.553 0.582 -0.596]
[-0.64 -0.626 0.445 0.64 0.626 -0.445]
[ 0.086 0.852 0.517 -0.086 -0.852 -0.517]
[-0.015 -0.999 -0.047 0.015 0.999 0.047]
[-0.211 -0.309 -0.928 0.211 0.309 0.928]
[ 0.111 -0.666 -0.738 -0.111 0.666 0.738]
[ 0.088 0.996 0.026 -0.088 -0.996 -0.026]
[-0.441 -0.75 0.493 0.441 0.75 -0.493]
[-0.787 0.129 0.603 0.787 -0.129 -0.603]
[-0.289 0.906 -0.308 0.289 -0.906 0.308]
[-0.465 0.545 0.697 0.465 -0.545 -0.697]
[ 0.152 -0.896 -0.418 -0.152 0.896 0.418]
[-0.503 -0.5 0.705 0.503 0.5 -0.705]
[-0.885 -0.122 -0.448 0.885 0.122 0.448]
[ 0.623 -0.367 -0.691 -0.623 0.367 0.691]
[ 0.017 0.972 0.233 -0.017 -0.972 -0.233]
[ 0.27 -0.639 0.721 -0.27 0.639 -0.721]
[-0.679 0.63 0.377 0.679 -0.63 -0.377]
[-0.219 -0.34 0.915 0.219 0.34 -0.915]
[ 0.173 0.679 0.713 -0.173 -0.679 -0.713]
[ 0.011 -0.875 0.484 -0.011 0.875 -0.484]
[-0.727 0.037 0.685 0.727 -0.037 -0.685]
[-0.389 0.619 0.682 0.389 -0.619 -0.682]
[ 0.059 0.531 0.845 -0.059 -0.531 -0.845]
[ 0.099 0.759 -0.644 -0.099 -0.759 0.644]
[-0.26 0.38 -0.888 0.26 -0.38 0.888]
[-0.123 -0.35 0.929 0.123 0.35 -0.929]
[-0.897 -0.077 -0.436 0.897 0.077 0.436]
[-0.761 -0.598 0.252 0.761 0.598 -0.252]
[-0.947 0.042 0.319 0.947 -0.042 -0.319]
[ 0.071 0.947 -0.315 -0.071 -0.947 0.315]
[-0.93 0.329 0.167 0.93 -0.329 -0.167]
```



```

[-0.547  0.537 -0.642  0.547 -0.537  0.642]
[ 0.937  0.014 -0.35  -0.937 -0.014  0.35 ]
[-0.644  0.489  0.588  0.644 -0.489 -0.588]
[ 0.746  0.547 -0.381 -0.746 -0.547  0.381]
[-0.688  0.704  0.176  0.688 -0.704 -0.176]
[ 0.597 -0.722  0.349 -0.597  0.722 -0.349]

```

(4) (Somewhat Difficult) Currently, in a 3D space, there are 8 quadrants numbered from 0 to 7. Instead of directly finding the quadrant, we will use 6 standard basis vectors and their inverse (6 vectors in total). The task is to find the "most similar" vector from this set.

After rotating the vectors, find the corresponding vector number and store it in a (100, 1) matrix. (Hint: Think about how to assign the vector number using the normalized vectors. You can avoid using the inner product for efficiency. If you're unsure, try considering the case in 2D and then apply the same idea to 3D. You can utilize the normalization step from part (2).)

```

# (4)
C_argmax = np.argmax(C, axis=1).reshape(-1,1)
print(C_argmax)

```

```

[[5]
 [2]
 [4]
 [2]
 [4]
 [1]
 [1]
 [4]
 [3]
 [1]
 [5]
 [0]
 [2]
 [1]
 [3]
 [3]
 [2]
 [1]
 [0]
 [3]
 [2]
 [3]
 [1]
 [4]
 [5]
 [5]
 [1]
 [4]
 [3]
 [1]
 [2]
 [4]
 [2]
 [3]
 [5]
 [1]
 [2]
 [3]
 [2]
 [2]
 [4]
 [3]
 [2]
 [2]
 [1]
 [5]
 [2]
 [3]
 [3]
 [3]
 [1]
 [3]
 [5]
 [0]
 [3]
 [0]
 [1]
 [4]

```

The 100 integers obtained here will be the values assigned to 100 vectors.

(5) (For those with extra time) For the 100 vectors, calculate the cosine of the angle θ between every pair of vectors. Verify that vectors that are "similar" (i.e., have a small angle between them) are likely to have been assigned the same integer value, while vectors that are "dissimilar" (i.e., have a large angle between them) are likely to have been assigned different integer values.

```
# (5)
D = B_normalized
D_num = C_argmax

#Setting a threshold for cosine similarity to determine similarity.
threshold = 1 / np.sqrt(2)

n_similar = 0          # Number of similar vectors
n_similar_same = 0     # Number of similar vectors with the same index
n_notsimilar = 0       # Number of dissimilar vectors
n_notsimilar_dif = 0   # Number of dissimilar vectors with different indices
for i in range(100):
    for j in range(i+1,100):
        cos = np.dot(D[i],D[j]) / (LA.norm(D[i]) * LA.norm(D[j]))
        # Similar vectors
        if cos > threshold:
            n_similar += 1
            if D_num[i] == D_num[j]:
                n_similar_same += 1
        # Dissimilar vectors
        else:
            n_notsimilar += 1
            if D_num[i] != D_num[j]:
                n_notsimilar_dif += 1

print('Proportion of similar vectors with the same integer value assigned: ', n_similar_same / n_similar)
print('Proportion of dissimilar vectors with different integer values assigned]: ', n_notsimilar_dif / n_notsimilar)

Proportion of similar vectors with the same integer value assigned:  0.5871559633027523
Proportion of dissimilar vectors with different integer values assigned]:  0.9106759016001911
```